

Multi-Class Classification on ACAC-4K Dataset

Syed Ali Haider
CSCI-SHU-360 Machine Learning
New York University Shanghai
sh6070@nyu.edu

Abstract—In this Kaggle project, I explored the development of an ensemble learning framework for image classification using the ACAC-4K dataset. Initially, I implemented a binary classification model to distinguish between AI-generated and non-AI images. However, this model misclassified over 1200 samples as AI-generated, prompting me to develop a "confusing data points cleaner" to filter ambiguous data points where the model wrongly predicted the label with high confidence. Additionally, I refined the dataset by removing blurry images and duplicates using Laplacian variance and perceptual hashing techniques. Multiple state-of-the-art architectures (ResNet18, ResNet50, ResNet152, DenseNet121, DenseNet201) were trained with Focal Loss to address class imbalance. Predictions were combined using weighted voting based on class-wise F1 scores. Hyperparameter tuning further improved individual model performance. The results demonstrate that ensemble learning combined with robust data cleaning and hyperparameter tuning significantly improves classification accuracy and robustness.

I. INTRODUCTION

Image classification is a fundamental problem in computer vision and while individual deep learning models have achieved remarkable success, they often struggle with noisy or ambiguous data. My approach attempts to address these challenges by combining ensemble learning with advanced data preprocessing techniques.

Initially, I implemented a binary classification model to distinguish between AI-generated and non-AI images. Despite achieving reasonable accuracy overall (around 80%), the model misclassified over 1200 images as AI-generated with high confidence. This highlighted the need for a more dynamic approach to handle noisy data. To address this issue, I developed a filtering function to identify and remove confusing data points that degraded model performance.

The key contributions of this project include:

- Preprocessing techniques to improve data quality by filtering blurry images and removing duplicates.
- Intensive hyper-parameter fine tuning.
- Development of a "confusing data points cleaner" to remove ambiguous samples.
- Training multiple deep learning models (ResNet18, ResNet50, ResNet152, DenseNet121, DenseNet201) using Focal Loss.
- Implementation of an ensemble learning framework using weighted voting based on class-wise F1 scores.

II. METHODOLOGY

A. Dataset Preprocessing

The ACAC-4K dataset contains high-resolution images labeled into multiple classes. The dataset posed challenges such as class imbalance and noisy labels.

To improve data quality:

- 1) **Blurry Image Detection:** Images were filtered based on Laplacian variance. Images with a variance below 90 were considered blurry and removed.
- 2) **Duplicate Removal:** Perceptual hashing (`dhash`) was applied to identify duplicate images. Only one instance of each duplicate group was retained.
- 3) **Confusing Data Points Cleaner:** High-confidence incorrect predictions (confidence > 0.5) from the initial binary classification model were identified during testing and removed from the training set.

After preprocessing, the dataset was split into training (90%) and testing (10%) sets. Here testing means my own local testing.

B. Model Architectures

I used five state-of-the-art architectures for my solution: ResNet18, ResNet50, ResNet152, DenseNet121, and DenseNet201. Each model's final layer was modified to match the number of classes in the dataset.

To address class imbalance during training, I used a custom implementation of Focal Loss:

$$L = -\alpha(1 - p_t)^\gamma \log(p_t) \quad (1)$$

where p_t is the probability of the true class, α is a weighting factor for classes, and γ reduces the loss contribution from easy examples.

The Adam optimizer was used with an initial learning rate of 5×10^{-5} . Data augmentation techniques such as random rotation, horizontal flipping, and color jittering were applied during training. To allow it to better generalise on unseen data.

C. Ensemble Learning

Ensemble learning combines predictions from multiple models to improve robustness and accuracy. For this project:

- 1) Class-wise F1 scores were calculated for each model on the validation set.
- 2) These scores were normalized to compute weights for each model.

- 3) For each test sample, weighted votes from all models were summed across classes:

$$V_c = \sum_{m=1}^M w_m P_m(c) \quad (2)$$

where V_c is the vote for class c , w_m is the weight for model m , and $P_m(c)$ is the predicted probability of class c by model m .

- 4) The class with the highest vote was selected as the final prediction.

For another part of the coding part, I was able to implement an ensemble model as well, but looking at the results I was able to understand,

D. Implementation

The training pipeline consisted of:

- 1) Training each model for 12 epochs using a batch size of 8.
- 2) Calculating training loss using Focal Loss.
- 3) Saving model checkpoints after training for evaluation and ensemble predictions.

After initial training:

- 1) High-confidence incorrect predictions were identified during testing using softmax probabilities.
- 2) These samples were filtered out from the training dataset using their indices.
- 3) The filtered dataset was used for retraining the models.

Predictions from individual models were generated using a custom test function that returned both predictions and sample indices.

E. Results

Each model's performance was evaluated on the test set:

TABLE I
INDIVIDUAL MODEL PERFORMANCE

Model	Accuracy (%)	Weighted F1 Score
ResNet18	51	0.43
ResNet50	55	0.46
ResNet152	57	0.48
DenseNet121	56	0.47
DenseNet201	58	0.49

The ensemble model achieved superior performance compared to individual models:

TABLE II
ENSEMBLE MODEL PERFORMANCE

Metric	Value
Accuracy (%)	62
Weighted F1 Score	0.54

A confusion matrix was plotted to analyze per-class performance. Classes with fewer samples had lower recall due to class imbalance.

III. HYPERPARAMETER TUNING AND RESULTS

Hyperparameter tuning plays a critical role in optimizing deep learning models for specific datasets. For this project, I conducted extensive experiments to identify the best combination of hyperparameters for training ResNet18, which ultimately achieved an accuracy of **71%** on the ACAC-4K dataset. Below, I outline the tuning process, key parameters adjusted, and their impact on performance.

1) *Tuning Process:* I systematically experimented with various hyperparameters to optimize model performance. The following parameters were considered during the tuning process:

- **Learning Rate:** Tested values included 1×10^{-3} , 5×10^{-4} , and 5×10^{-5} . A lower learning rate (5×10^{-5}) provided stable convergence and reduced overfitting.
- **Batch Size:** Batch sizes of 4, 8, and 16 were evaluated. A batch size of 4 was selected as it balanced memory constraints and gradient stability.
- **Loss Function:** The Focal Loss function was used with $\alpha = 1.0$ and $\gamma = 2.0$ to address class imbalance by focusing on harder-to-classify samples.
- **Data Augmentation:** Techniques such as random horizontal flipping, color jittering (brightness, contrast, saturation adjustments), and normalization were applied to improve generalization.
- **Number of Epochs:** Models were trained for up to 12 epochs to ensure sufficient learning without overfitting.

The experiments were conducted using a fixed random seed (SEED=42) to ensure reproducibility.

2) *Training Results:* The model was trained on the ACAC-4K dataset using the tuned hyperparameters. The training loss decreased steadily across epochs, demonstrating effective learning. Table III summarizes the loss values at each epoch.

TABLE III
TRAINING LOSS ACROSS EPOCHS FOR RESNET18

Epoch	Training Loss
1	1.4738
2	1.3902
3	1.3292
4	1.3040
5	1.2729
6	1.2449
7	1.2264
8	1.2002
9	1.1758
10	1.1419
11	1.1169
12	1.0975

3) *Evaluation on Test Set:* After training, the model was evaluated on the test set using accuracy and F1 score metrics. The predictions were saved in a CSV file containing both sample indices and predicted labels for further analysis.

The final model achieved:

- Accuracy: **71%**

4) *Impact of Hyperparameter Tuning*: The hyperparameter tuning process significantly improved model performance compared to baseline settings:

- 1) The use of Focal Loss effectively addressed class imbalance by penalizing easy-to-classify samples and emphasizing harder ones.
- 2) A lower learning rate (5×10^{-5}) provided smoother convergence and reduced oscillations in training loss.
- 3) Data augmentation techniques improved generalization by exposing the model to diverse variations of training samples.
- 4) Smaller batch sizes ensured stable gradient updates while accommodating GPU memory constraints.

These optimizations collectively enabled ResNet18 to achieve good results on this dataset.

IV. DISCUSSION AND CONCLUSION

A. Discussion

In this project, I explored two distinct approaches to improve the classification performance on the ACAC-4K dataset: ensemble learning and hyperparameter tuning. Both approaches demonstrated promising results, showcasing their potential to address the challenges posed by noisy data, class imbalance, and dataset complexity.

a) *Ensemble Learning*:: The ensemble learning approach combined predictions from multiple deep learning architectures (ResNet18, ResNet50, ResNet152, DenseNet121, DenseNet201) using a weighted voting mechanism. By leveraging class-wise F1 scores to assign weights, this method accounted for the varying strengths of individual models across classes. The ensemble achieved an accuracy of **62%**, a significant improvement over the baseline performance of **55%** achieved by individual models.

This improvement highlights the robustness of ensemble learning in capturing diverse feature representations and decision boundaries. However, the approach required substantial computational resources due to the need to train multiple models independently. Additionally, while ensemble learning improved overall accuracy and robustness, it did not directly address noisy or ambiguous data points in the test set.

b) *Hyperparameter Tuning*:: The hyperparameter tuning process focused on optimizing key parameters for ResNet18. Adjustments included:

- Reducing the learning rate to 5×10^{-5} for smoother convergence.
- Using Focal Loss with $\alpha = 1.0$ and $\gamma = 2.0$ to handle class imbalance.
- Applying data augmentation techniques such as random horizontal flipping and color jittering to improve generalization.
- Reducing batch size to 4 to balance memory constraints and gradient stability.

This approach achieved an accuracy of **71%** on Kaggle using a simple yet effective model configuration. The results

demonstrate that careful tuning of hyperparameters can significantly enhance model performance even without complex ensembling techniques. However, this method was limited by its reliance on a single architecture (ResNet18), which may not fully capture the diversity of features present in the dataset.

c) *Comparison of Approaches*:: Both approaches showed clear strengths:

- 1) Ensemble learning improved accuracy from 55% to 62%, demonstrating its ability to combine complementary strengths of multiple models.
- 2) Hyperparameter tuning further boosted accuracy to 71%, showcasing how careful optimization can enhance a single model's performance.

Despite these successes, I was unable to combine both approaches due to time constraints during the final week of this project. Long queue times on high-performance computing (HPC) resources and high training times further limited my ability to integrate hyperparameter-tuned models into an ensemble framework.

B. Conclusion

This project demonstrated that both ensemble learning and hyperparameter tuning are effective strategies for improving image classification performance on challenging datasets like ACAC-4K. Ensemble learning provided robustness by aggregating predictions from multiple architectures, while hyperparameter tuning optimized a single model's performance through careful adjustments.

If I had more time and computational resources, I hypothesize that combining these two approaches could yield even better results. Specifically:

- Training multiple models with optimized hyperparameters could enhance individual model performance.
- Integrating these tuned models into an ensemble framework could potentially achieve an accuracy of around **75–80%**, leveraging both optimization strategies simultaneously.

Future work should focus on:

- 1) Automating hyperparameter search using tools like Optuna or Ray Tune for more exhaustive exploration.
- 2) Incorporating pretrained weights from ImageNet or other large-scale datasets to accelerate convergence.
- 3) Exploring advanced ensembling techniques such as stacking or blending for further improvements in robustness.
- 4) Addressing computational bottlenecks by utilizing distributed training or more efficient hardware resources.

In conclusion, this project highlights the importance of combining complementary strategies in machine learning workflows. By integrating ensemble learning with hyperparameter tuning, we can unlock new levels of accuracy and robustness in image classification tasks.